DBMS — EXAM-ORIENTED QUESTION & ANSWER SHEET

EXPANDED MODEL ANSWERS | CEC305 | D Y Patil University

This edition expands the answers for actual exam writing. Every entry is presented as a question, marks, and a model answer with full explanations, examples, SQL, or step-by-step working where required. The priority follows the supplied previous-year question analysis.

Answer-length guide: For 5 marks, include a definition, four or five explained points and an example where relevant. For 10 marks, include the complete procedure, working, diagram/schema or code and a conclusion.

Priority	Topics
MUST-DO	Normalization; Relational Algebra; ER Mapping; Functional Dependency; SQL Queries; Serializability; ER Design; Triggers; DDL/DML
HIGH	Transactions and ACID; Generalization/Specialization; SQL Constraints
MEDIUM	Locking/2PL/Deadlock; SQL Joins; Stored Procedures; DBMS Architecture; SQL vs PL/SQL; ER Attributes; Aggregate Functions
LOW	Recovery; Transaction States; Cursors/Functions; Participation; DBA; Super/Candidate Keys

The first nine ranked topics appeared in all four papers analysed in the supplied question bank. Lower-frequency topics are still included at the end for coverage.

PART I — MUST-DO TOPICS

1. Normal Forms and Normalization

Q1. Define normalization. Explain why it is needed and describe First Normal Form (1NF) with an example.

Marks: 5 marks

Answer:

Definition: Normalization is the systematic process of organizing data into well-structured relations to reduce unnecessary duplication and undesirable dependencies. It improves data integrity and makes a database easier to maintain.

Why normalization is required:

- It reduces repeated storage of the same facts and prevents inconsistent copies of data.
- It prevents an **update anomaly**, where the same fact must be changed in several rows.
- It prevents an **insertion anomaly**, where a fact cannot be inserted unless an unrelated fact is also supplied.
- It prevents a **deletion anomaly**, where deleting one record unintentionally removes other useful information.

First Normal Form (1NF): A relation is in 1NF when every cell contains one atomic (indivisible) value, there are no repeating groups or multivalued cells, and each row can be uniquely identified.

Example: STUDENT(Roll, Name, Phone) is not in 1NF if Roll=1, Name=Amit, Phone="98201, 98202" stores two numbers in one cell. Convert it to STUDENT(Roll, Name) and STUDENT_PHONE(Roll, Phone). The phone table stores one number per row and can use (Roll, Phone) as its composite key. Every cell is then atomic and the repeating group is removed.

PYQ coverage: A Q1c; B Q1d; D Q2A-iii

Exam tip: Use the terms atomic values, repeating groups and a before/after table example.

Q2. Explain 1NF, 2NF and 3NF with suitable examples.

Marks: 5 marks

Answer:

1NF: A relation is in First Normal Form if each attribute contains a single atomic value and there are no repeating groups. If several phone numbers are stored in one STUDENT.Phone cell, the table violates 1NF. Create STUDENT_PHONE(Roll, Phone) to store one phone number per row.

2NF: A relation must be in 1NF and each non-prime attribute must depend on the whole candidate key, not only part of a composite key. Example: TEACHER(Teacher_ID, Subject, Teacher_Age), key (Teacher_ID, Subject). If Teacher_ID -> Teacher_Age, age depends on only part of the key. Decompose into TEACHER_DETAIL(Teacher_ID, Teacher_Age) and TEACHER_SUBJECT(Teacher_ID, Subject).

3NF: A relation must be in 2NF and must not contain a transitive dependency of a non-prime attribute on a candidate key. Example: EMPLOYEE(Emp_ID, Emp_Name, Zip, City), where Emp_ID -> Zip and Zip -> City. City is transitively dependent on Emp_ID. Decompose into EMPLOYEE(Emp_ID, Emp_Name, Zip) and ZIP_DETAIL(Zip, City).

Conclusion: 1NF removes non-atomic values; 2NF removes partial dependencies; 3NF removes transitive dependencies. These steps reduce redundancy and help prevent insertion, update and deletion anomalies.

PYQ coverage: D Q2A-iii

Q3. Explain lossy and lossless decomposition with an example.

Marks: 5 marks

Answer:

Lossless decomposition: a relation is decomposed losslessly when joining the smaller relations reconstructs exactly the original relation, without losing original tuples or generating extra tuples.

Lossy decomposition: a decomposition is lossy when the join creates spurious (false) tuples or original associations cannot be recovered correctly.

Example: Let R(A,B,C) contain (1,x,p) and (2,x,q). Decompose into R1(A,B)={(1,x),(2,x)} and R2(B,C)={(x,p),(x,q)}. Joining R1 and R2 on B gives four tuples: (1,x,p), (1,x,q), (2,x,p) and (2,x,q). The tuples (1,x,q) and (2,x,p) were not in R, so the decomposition is lossy.

Lossless condition: for a binary decomposition into R1 and R2, a common sufficient condition is that the common attributes functionally determine all attributes of at least one of the decomposed relations; effectively, the common attributes form a superkey of one side.

Conclusion: a sound normalization decomposition should be lossless and, where possible, preserve the original functional dependencies.

PYQ coverage: B Q2d

Q4. Explain Fifth Normal Form (5NF) and apply it to a student database.

Marks: 5 marks

Answer:

Definition: Fifth Normal Form, also called Project-Join Normal Form, is reached when every non-trivial join dependency in a relation is implied by its candidate keys. In practical terms, the relation cannot be decomposed further to remove redundancy through a join dependency unless that dependency is already implied by the keys.

Purpose: 5NF handles redundancy caused by complex join dependencies, particularly when three or more independent relationships create repeated combinations. It addresses issues beyond the common dependencies handled by 1NF, 2NF and 3NF.

Example: Consider R(Student, Course, Instructor), recording which student takes which course with which instructor. If the business rules guarantee that the information can be represented through three related facts, decompose into:

- R1(Student, Course)
- R2(Course, Instructor)
- R3(Student, Instructor)

When the stated join dependency holds, the original relation is reconstructed by joining R1, R2 and R3. The decomposition can reduce repeated combinations of student, course and instructor facts.

Important condition: the business rules must guarantee that the join reconstructs the original relation. A decomposition is not automatically lossless simply because it creates smaller tables.

PYQ coverage: C Q1c

2. Relational Algebra Operations

Q5. Explain selection and projection in relational algebra with suitable examples.

Marks: 5 marks

Answer:

Relational algebra is a procedural query language: an expression specifies the operations used to obtain the required relation.

Selection (sigma): chooses rows (tuples) satisfying a condition. It is written sigma_condition(R). Example: sigma_Age>18(STUDENT) returns only the students older than 18. Selection is similar to the WHERE clause in SQL.

Projection (pi): chooses specified columns (attributes). It is written pi_attribute-list(R). Example: pi_Roll,Name(STUDENT) returns only the Roll and Name columns. In classical relational algebra, duplicate result tuples are removed.

Combined expression: pi_Name(sigma_Age>18(STUDENT)) first selects students older than 18 and then displays only their names. The inner operation is performed before the outer operation.

Difference: selection filters rows based on their values, whereas projection selects columns. Both are unary operations because each takes one relation as input.

PYQ coverage: A Q1b; B Q5b

Exam tip: Write operator, meaning, syntax and example for each operation.

Q6. Explain UNION, SET DIFFERENCE, INTERSECTION and CARTESIAN PRODUCT with examples.

Marks: 5 marks

Answer:

These are binary relational algebra operations because they take two relations as input.

Union (R union S): returns tuples present in R, S or both. R and S must be union-compatible: they must have the same number of attributes and corresponding attributes must have compatible domains. Example: STUDENT union EMPLOYEE lists tuples in either relation.

Set difference (R - S): returns tuples present in R but not in S. Example: STUDENT - EMPLOYEE returns student tuples absent from EMPLOYEE.

Intersection (R intersect S): returns tuples common to both relations. Example: pi_Customer(borrower) intersect pi_Customer(depositor) finds customers who have both a loan and an account.

Cartesian product (R x S): pairs every tuple of R with every tuple of S. If R has m tuples and S has n, the result has m*n tuples. STUDENT x COURSE pairs every student with every course; a selection condition can then be applied to form a meaningful join.

Conclusion: Union, difference and intersection require union compatibility; Cartesian product does not, but can create many rows.

PYQ coverage: A Q3b; B Q3b; D Q3i

Q7. Explain unary operations in relational algebra and construct queries.

Marks: 10 marks

Answer:

A unary operation operates on one relation. The main unary operations are selection, projection and rename.

Selection: selects rows satisfying a condition; syntax sigma_condition(R). **Projection:** selects specified columns; syntax pi_A1,A2(R). **Rename:** renames a relation or its attributes; syntax rho_NewName(R).

Assume STUDENT(Roll, Name, Age, City). The expressions below illustrate each operation:

```
/* Students older than 18 */
sigma_Age>18(STUDENT)

/* Show only roll number and name */
pi_Roll,Name(STUDENT)

/* Give the relation a new name */
rho_STUDENT1(STUDENT)

/* Names of students older than 18 */
pi_Name(sigma_Age>18(STUDENT))
```

In the final expression, selection is performed first to filter student rows; projection is then applied to keep only Name. Selection is therefore a row operation, projection is a column operation, and rename changes relation or attribute names.

Importance: these operators form the basic building blocks for more complex relational queries and are the theoretical basis for SQL query processing.

PYQ coverage: B Q5b

Exam tip: Show syntax and an actual expression for each operation.

Q8. Explain JOIN and DIVISION in relational algebra with examples.

Marks: 5 marks

Answer:

Join: combines related tuples from two relations using a matching condition. It can be viewed as a Cartesian product followed by selection. If STUDENT(S_ID, Name, C_ID) and COURSE(C_ID, C_Name) share C_ID, STUDENT join COURSE returns students with their course names.

Natural join matches all same-named attributes and keeps one copy of the common column. **Theta join** uses a comparison condition such as <, > or =. **Equi-join** is a theta join using equality. **Outer joins** preserve unmatched rows, filling missing-side attributes with NULL.

Division: R(A,B) divided by S(B) returns A values associated with every B value in S. Example: TAKES(Student, Course) divided by REQUIRED(Course) returns students who have taken every required course.

Use: joins combine related information, while division answers “for all” queries, such as which students completed all compulsory courses.

PYQ coverage: C Q1b; A Q3b

---

3. ER-to-Relational Schema Mapping

Q9. Explain the rules for converting an ER diagram into a relational schema.

Marks: 5 marks

Answer:

ER-to-relational mapping converts entities, attributes and relationships into tables, columns, primary keys and foreign keys.

- **Strong entity:** create a table; simple attributes become columns and the key becomes the primary key.
- **Composite attribute:** store its component attributes separately, e.g. Address becomes Street, City and State.
- **Multivalued attribute:** create a separate table with the owner key and multivalued attribute; their combination normally forms the primary key.
- **Derived attribute:** normally calculate it when required, e.g. Age from Date_of_Birth.
- **Weak entity:** include the owner primary key and the weak entity partial key; together they identify the row.
- **1:1 relationship:** place one side’s key as a foreign key in the other, preferably on the total-participation side.
- **1:N relationship:** put the primary key of the 1-side in the N-side table as a foreign key.
- **M:N relationship:** create a separate relation containing both entity keys and any relationship attributes.
- **Specialization/generalization:** commonly create a superclass table and subclass tables whose keys also reference the superclass.

PYQ coverage: A Q2b; D Q2A-i

Exam tip: A 5-mark answer should systematically cover entities, multivalued attributes and relationship cardinalities.

Q10. Illustrate how 1:1 and M:N relationships are mapped into relational schemas.

Marks: 5 marks

Answer:

One-to-one (1:1): Suppose PERSON(Per_ID, Name) has a HAS relationship with PASSPORT(Pass_No). Each person has at most one passport and each passport belongs to one person. Place Pass_No as a unique foreign key in PERSON or Per_ID as a unique foreign key in PASSPORT. The foreign key is normally placed on the side with total participation. If merging is appropriate, PERSON(Per_ID, Name, Pass_No) may be used.

Many-to-many (M:N): Suppose STUDENT(S_ID, Name) ENROLLS in COURSE(C_ID, Title). One student can take many courses and each course can contain many students. Neither entity table alone represents all pairings without repetition. Create a separate relation:

```
STUDENT(S_ID PK, Name)
COURSE(C_ID PK, Title)
ENROLLS(S_ID FK, C_ID FK, Grade,
        PRIMARY KEY (S_ID, C_ID))
```

S_ID and C_ID reference their parent tables. Grade belongs in ENROLLS because it describes a student’s enrolment in a particular course, not the student or course independently. The composite key prevents the same student-course enrolment from being duplicated.

PYQ coverage: B Q2b

---

4. Functional Dependency and Normalization Problems

Q11. Define functional dependency. Explain its types and significance in normalization.

Marks: 5 marks

Answer:

Definition: a functional dependency (FD) X -> Y means each value of attribute set X is associated with exactly one value of attribute set Y. X is the determinant and Y is the dependent. If two rows agree on X, they must also agree on Y.

Example: In STUDENT(Roll_No, Name, Dept_Name, Dept_Building), Roll_No -> Name is valid if each roll number identifies one student. Dept_Name -> Dept_Building is valid if each department has one building.

Types:

- **Trivial:** Y is a subset of X, e.g. (Roll_No, Name) -> Name.
- **Non-trivial:** Y is not a subset of X, e.g. Roll_No -> Name.
- **Partial:** a non-key attribute depends on only part of a composite key.
- **Transitive:** X -> Y and Y -> Z imply X -> Z, e.g. Emp_ID -> Dept_ID and Dept_ID -> Dept_Name.

Significance: FDs help identify candidate keys and detect partial or transitive dependencies. They determine whether a table satisfies 2NF, 3NF or BCNF, guide decomposition, reduce redundancy, prevent anomalies and support integrity.

PYQ coverage: A Q2c; B Q3e; D Q1ii

Q12. Explain non-trivial and transitive functional dependencies with examples.

Marks: 5 marks

Answer:

Non-trivial FD: X -> Y is non-trivial when Y is not contained in X. For example, Roll_No -> Name is non-trivial because Name is not part of Roll_No. (Roll_No, Name) -> Name is trivial because Name is already in the left-hand side.

Transitive FD: this occurs when X determines Y and Y determines Z, giving an indirect dependency X -> Z. Example:

```
Emp_ID -> Dept_ID
Dept_ID -> Dept_Name
Therefore: Emp_ID -> Dept_Name
```

Here Emp_ID indirectly determines Dept_Name through Dept_ID. If Dept_ID is not a candidate key and Dept_Name is a non-prime attribute, storing all three attributes in EMPLOYEE can repeat department names in many rows.

Relation to normalization: 2NF removes partial dependencies on part of a composite key. 3NF addresses transitive dependencies of non-prime attributes on candidate keys. A suitable decomposition of the example is EMPLOYEE(Emp_ID, Dept_ID) and DEPARTMENT(Dept_ID, Dept_Name), so department information is stored once.

PYQ coverage: B Q4b; D Q1ii

Q13. Given R(A,B,C,D) and F={AB -> CD, B -> C}, determine whether R is in 2NF. If not, convert it into 2NF.

Marks: 10 marks

Answer:

Step 1 — Candidate key: From AB -> CD, attributes A and B determine C and D. Therefore (AB)+ = {A,B,C,D}. A+ = {A}, and B+ = {B,C}, so neither A nor B alone is a key. AB is the candidate key.

Step 2 — Prime attributes: A and B are prime attributes because they are part of candidate key AB. C and D are non-prime attributes.

Step 3 — Test 2NF: 2NF requires 1NF and no partial dependency of a non-prime attribute on a proper subset of a candidate key. The FD B -> C shows that non-prime attribute C depends on B alone, a proper subset of AB. Thus R is not in 2NF.

Step 4 — Decomposition:

```
R1(B, C)       Primary key: B
R2(A, B, D)    Primary key: (A, B)
```

R1 represents B -> C. R2 retains AB -> D. Neither relation contains a non-prime attribute depending on only part of its candidate key, so both satisfy 2NF.

Step 5 — Check: the common attribute B is a key of R1, so the decomposition is lossless. B -> C is preserved in R1 and AB -> D in R2; together they preserve AB -> CD. Therefore the decomposition is lossless and dependency-preserving.

PYQ coverage: C Q3b

Exam tip: Write closure, candidate key, prime attributes, violation, decomposition and lossless check.

Q14. Explain non-trivial FD and normalize the Faculty/Course table into an appropriate normal form.

Marks: 10 marks

Answer:

Given FACULTY_COURSE(F_ID, F_Address, Course_ID, Course_Name):

```
F_ID  F_Address  Course_ID  Course_Name
101   Delhi      C1         MCA
101   Delhi      C2         MBA
102   Noida      C1         MCA
102   Noida      C2         MBA
```

Non-trivial FD: X -> Y is non-trivial when Y is not a subset of X. In this table, F_ID -> F_Address and Course_ID -> Course_Name are non-trivial FDs.

Candidate key: assume (F_ID, Course_ID) identifies each faculty-course pairing. All values are atomic, so the relation satisfies 1NF.

2NF violation: F_Address depends on F_ID alone, which is only part of the composite key. Course_Name depends on Course_ID alone, also part of the composite key. These partial dependencies mean the relation is not in 2NF.

Decomposition:

```
FACULTY(F_ID, F_Address)
  Primary key: F_ID
COURSE(Course_ID, Course_Name)
  Primary key: Course_ID
TEACHES(F_ID, Course_ID)
  Primary key: (F_ID, Course_ID)
```

FACULTY stores faculty details once; COURSE stores course details once; TEACHES records which faculty teaches each course. The decomposition removes the partial dependencies. Under the stated dependencies, the decomposed relations also satisfy 3NF/BCNF.

PYQ coverage: B Q4b

5. SQL Query-Writing Problems

Q15. For ALLSTUDENT(ROLLNO, FULLNAME, ADDRESS, COURSE, MARKS), write SQL for the four required queries.

Marks: 10 marks

Answer:

The requirements are: (i) students older than 19, (ii) marks greater than 75%, (iii) names beginning with S, and (iv) students living in Andheri.

```
-- (i) Assumption: AGE column is available
SELECT * FROM ALLSTUDENT
WHERE AGE > 19;

-- If DOB is stored instead (Oracle)
SELECT * FROM ALLSTUDENT
WHERE MONTHS_BETWEEN(SYSDATE, DOB)/12 > 19;

-- (ii) Marks greater than 75 out of 100
SELECT FULLNAME FROM ALLSTUDENT
WHERE MARKS > 75;

-- (iii) Names beginning with S
SELECT FULLNAME FROM ALLSTUDENT
WHERE FULLNAME LIKE 'S%';

-- (iv) Address contains Andheri
```

```
SELECT * FROM ALLSTUDENT
WHERE ADDRESS LIKE '%Andheri%';
```

Explanation: AGE > 19 excludes students aged exactly 19. If Date of Birth is used instead, age is calculated from the current date. MARKS > 75 returns only scores above 75; use >= 75 if exactly 75 should qualify. LIKE S% matches names beginning with S, because % means any sequence of characters. LIKE %Andheri% searches for the locality anywhere in the address; use ADDRESS = Andheri if the field stores only the locality.

Schema note: the supplied schema has no AGE or DOB column, so state the assumption for part (i).

PYQ coverage: A Q3a

Q16. For Employee, Works, Company and Manages, write SQL for the five queries in the question.

Marks: 10 marks

Answer:

Schema: Employee(empname, street, city, date_of_joining); Works(empname, company_name, salary); Company(company_name, city); Manages(empname, manager_name).

```
-- (i) Change John's city
UPDATE Employee SET city='Mumbai'
WHERE empname='John';

-- (ii) Employees who joined in October (Oracle)
SELECT * FROM Employee
WHERE EXTRACT(MONTH FROM date_of_joining)=10;

-- (iii) Raise salaries at TCS by 10%
UPDATE Works SET salary=salary*1.10
WHERE company_name='TCS Corporation';

-- (iv) Employees living in the same city as their company
SELECT E.empname
FROM Employee E, Works W, Company C
WHERE E.empname=W.empname
 AND W.company_name=C.company_name
 AND E.city=C.city;

-- (v) Employees above their company's average salary
SELECT W.empname FROM Works W
WHERE W.salary > (SELECT AVG(W2.salary)
 FROM Works W2
 WHERE W2.company_name=W.company_name);
```

Explanation: UPDATE modifies matching rows, so a WHERE clause is essential. EXTRACT(MONTH...) selects joining dates in October. Multiplying salary by 1.10 gives a 10% increase. The fourth query joins employee, employment and company data, then compares the cities. The fifth query is a correlated subquery: it computes the average for the same company as the outer Works row and returns employees above that average.

Assumption: the schema has no salary-year column, so the salary update applies to all listed Works rows for TCS Corporation.

PYQ coverage: B Q4a

Q17. Write SQL queries for the First Bank Corporation database as specified in the question.

Marks: 10 marks

Answer:

Schema: Employee(employee_name, street, city), Works(employee_name, company_name, salary), Company(company_name, city), Manages(employee_name, manager_name).

```
-- (i) Names of employees working for First Bank
SELECT employee_name FROM Works
WHERE company_name='First Bank Corporation';

-- (ii) Names and cities of those employees
SELECT E.employee_name, E.city
FROM Employee E JOIN Works W
 ON E.employee_name=W.employee_name
WHERE W.company_name='First Bank Corporation';

-- (iii) Name, street and city with salary > 10000
SELECT E.employee_name, E.street, E.city
```

```
FROM Employee E JOIN Works W
 ON E.employee_name=W.employee_name
WHERE W.company_name='First Bank Corporation'
 AND W.salary>10000;

-- (iv) Employees not working for First Bank
SELECT E.employee_name FROM Employee E
WHERE E.employee_name NOT IN (
 SELECT W.employee_name FROM Works W
 WHERE W.company_name='First Bank Corporation');

-- (v) Average salary
SELECT AVG(salary) AS average_salary FROM Works;
```

The first query filters Works by company. The next two join Employee to Works to retrieve residence details and add the salary restriction. The fourth uses a subquery to exclude employees who work for First Bank Corporation. The fifth uses AVG to calculate the average salary in Works.

PYQ coverage: C Q3c

Q18. For Employee(Emp_no, Name, Emp_city) and Company(Emp_no, Company_name, Salary), write the required SQL queries.

Marks: 10 marks

Answer:

```
-- (a) Employee name and company name
SELECT E.Name, C.Company_name
FROM Employee E JOIN Company C
 ON E.Emp_no=C.Emp_no;

-- (b) Employees earning more than 10000
SELECT E.Name, E.Emp_city, C.Company_name, C.Salary
FROM Employee E JOIN Company C
 ON E.Emp_no=C.Emp_no
WHERE C.Salary>10000;

-- (c) Employees in XYZ company
SELECT E.Name
FROM Employee E JOIN Company C
 ON E.Emp_no=C.Emp_no
WHERE C.Company_name='XYZ';
```

Emp_no is the linking key between employee details and the employment/company data. The first query displays employee and company names. The second adds city and salary and filters on salary. The third restricts the output to the company name XYZ.

PYQ coverage: D Q2B-ii

---

6. Conflict and View Serializability

Q19. Explain serializability and conflict serializability using a precedence graph.

Marks: 5 marks

Answer:

Schedule: a schedule is an ordering of operations from one or more transactions. In a serial schedule, one transaction finishes before the next starts. In a non-serial schedule, transaction operations are interleaved. A schedule is serializable if its effect equals that of some serial schedule.

Conflicts: two operations conflict if they belong to different transactions, access the same data item, and at least one is a write. Thus read-write, write-read and write-write pairs conflict; read-read pairs do not.

Precedence graph test:

- Create one node for every transaction.
- For each conflicting pair, if Ti's operation occurs before Tj's, draw an edge Ti -> Tj.
- If the graph has no cycle, the schedule is conflict serializable; a topological ordering gives the equivalent serial order.
- If the graph has a cycle, the schedule is not conflict serializable.

For example, edges T1 -> T2 and T2 -> T1 form a cycle. There is no serial ordering that satisfies both ordering constraints, so the schedule is not conflict serializable.

PYQ coverage: B Q5c; C Q2f; D Q2B-i

Q20. Explain view serializability and distinguish it from conflict serializability.

Marks: 10 marks

Answer:

View serializability: schedule S is view serializable if it is view-equivalent to some serial schedule S'. Two schedules are view-equivalent if they preserve the same initial reads, read-from relationships and final writes.

Conditions for view equivalence:

- **Initial read:** if a transaction reads the initial value of an item in S, it must do so in S'.
- **Read-from relationship:** if Ti reads a value written by Tj in S, it must read the value written by Tj in S'.
- **Final write:** the transaction performing the final write of each item must be the same in both schedules.

Conflict serializability: tested by creating a precedence graph from conflicting operation pairs. If the graph is acyclic, the schedule is conflict serializable. This is more restrictive than view serializability.

Basis	Conflict serializability	View serializability
Test	Precedence graph	Initial reads, read-from and final writes
Scope	More restrictive	More general
Relationship	Every conflict-serializable schedule is view serializable	Some view-serializable schedules are not conflict serializable
Special case	Cycle means it fails the test	May allow certain schedules with blind writes

Conclusion: a conflict-serializable schedule is always view serializable, but the reverse is not always true. A schedule with a cycle in its precedence graph may still be view serializable if it preserves the view-equivalence conditions, commonly involving blind writes.

PYQ coverage: B Q5c; D Q2B-i

7. ER Diagram Design and EER

Q21. Design an ER model for a university database containing Student, Course, Professor and Department.

Marks: 5 marks

Answer:

Entities and attributes:

- STUDENT(Roll_No, Name, DOB, Age), where Age can be derived from DOB.
- COURSE(Course_ID, Title, Credits).
- PROFESSOR(Prof_ID, Name, Phone), where Phone may be multivalued.
- DEPARTMENT(Dept_ID, Dept_Name, Location).

Relationships and cardinalities:

- STUDENT ENROLLS in COURSE: M:N, because one student may take many courses and each course may contain many students.
- PROFESSOR TEACHES COURSE: 1:N, assuming one professor may teach many courses and a course is assigned one professor.
- PROFESSOR WORKS_FOR DEPARTMENT: N:1, because many professors may belong to one department.
- STUDENT BELONGS_TO DEPARTMENT: N:1, because many students belong to one department.

Relational conversion: create a table for each entity. For M:N enrollment, create ENROLLS(Roll_No, Course_ID) with both attributes as foreign keys and their combination as the primary key. For 1:N relationships, put the key of the 1-side in the N-side table.

In a hand-drawn diagram use rectangles for entities, ellipses for attributes, diamonds for relationships, underline key attributes, and label cardinalities.

PYQ coverage: A Q2a

Q22. Describe a Hospital EER model and convert it to a relational schema.

Marks: 10 marks

Answer:

Entities and relationships: PATIENT is admitted to one WARD (N:1). DOCTOR treats PATIENT (M:N). DOCTOR is specialized into SURGEON and PHYSICIAN. PATIENT has multiple test records represented by weak entity TEST_LOG, identified through PATIENT. PATIENT may also have multiple phone numbers.

Relational schema:

```
PATIENT(Patient_ID PK, Name, DOB, Ward_No FK)
PATIENT_PHONE(Patient_ID FK, Phone,
              PK (Patient_ID, Phone))
DOCTOR(Doctor_ID PK, Name, Specialty)
SURGEON(Doctor_ID PK/FK, ...)
PHYSICIAN(Doctor_ID PK/FK, ...)
WARD(Ward_No PK, Type)
TREATED_BY(Patient_ID FK, Doctor_ID FK,
          PK (Patient_ID, Doctor_ID))
TEST_LOG(Patient_ID FK, Test_No, Test_Type,
         Result, Test_Date,
         PK (Patient_ID, Test_No))
```

Mapping explanation: Ward_No is placed in PATIENT because admission is N:1. PATIENT_PHONE is separate because Phone is multivalued. TREATED_BY is separate because treatment is M:N and the pair of foreign keys identifies a patient-doctor pairing. SURGEON and PHYSICIAN inherit the identifier from DOCTOR, so Doctor_ID is both their PK and FK. TEST_LOG includes Patient_ID with Test_No because it is a weak entity dependent on a patient.

In the diagram, show an ISA/specialization link between DOCTOR and its subclasses. Mark the specialization disjoint only if the stated business rule says a doctor cannot belong to both subclasses.

PYQ coverage: B Q5a

Q23. Construct an ER model for a Company database and convert it into relational tables.

Marks: 10 marks

Answer:

Entities: EMPLOYEE(Emp_ID, Name, Salary, Address), DEPARTMENT(Dept_No, Dept_Name, Locations), PROJECT(Proj_No, Proj_Name, Location), and DEPENDENT(Dep_Name, Relation, Age). DEPENDENT is a weak entity owned by EMPLOYEE; department locations may be multivalued.

Relationships: Employee works for Department (N:1); Department controls Project (1:N); Employee works on Project (M:N) with relationship attribute Hours; Employee has Dependent (1:N identifying relationship).

Relational schema:

```
EMPLOYEE(Emp_ID PK, Name, Salary, Address, Dept_No FK)
DEPARTMENT(Dept_No PK, Dept_Name)
DEPT_LOCATIONS(Dept_No FK, Location,
               PK (Dept_No, Location))
PROJECT(Proj_No PK, Proj_Name, Location, Dept_No FK)
WORKS_ON(Emp_ID FK, Proj_No FK, Hours,
         PK (Emp_ID, Proj_No))
DEPENDENT(Emp_ID FK, Dep_Name, Relation, Age,
         PK (Emp_ID, Dep_Name))
```

Mapping explanation: Dept_No is placed in EMPLOYEE for the N:1 relation. Dept_No is placed in PROJECT for the 1:N relation. WORKS_ON is separate because employee-project assignment is M:N and Hours describes the assignment. DEPT_LOCATIONS represents the multivalued location attribute. DEPENDENT includes the owner key and partial key to identify a dependent in relation to an employee.

PYQ coverage: C Q3a

Q24. Design an ER model for a car-insurance company and construct its tables.

Marks: 10 marks

Answer:

Entities: PERSON(Driver_ID, Name, Address), CAR(Licence, Model, Year) and ACCIDENT(Report_No, Acc_Date, Location). Their keys are Driver_ID, Licence and Report_No respectively.

Relationships: PERSON OWNS CAR is 1:N: one person may own several cars and each car has one owner. CAR PARTICIPATES_IN ACCIDENT is M:N: one car may have many accidents and one accident may involve multiple cars. Car participation in accidents is partial because a car may have zero accidents. Damage_Amount belongs to the participation relationship.

Relational schema:

```
PERSON(Driver_ID PK, Name, Address)
CAR(Licence PK, Model, Year, Driver_ID FK)
ACCIDENT(Report_No PK, Acc_Date, Location)
PARTICIPATED(Licence FK, Report_No FK,
            Damage_Amount,
            PK (Licence, Report_No))
```

Explanation: Driver_ID is placed in CAR for the 1:N ownership relationship. The M:N accident relationship requires PARTICIPATED so multiple cars can be associated with one accident and a car can be involved in many accidents. Damage_Amount is stored in PARTICIPATED because damage can differ by car in a given accident.

A diagram should show PERSON -- OWNS -- CAR with 1:N cardinality and CAR -- PARTICIPATED_IN -- ACCIDENT with M:N cardinality.

PYQ coverage: D Q3ii-a

8. Triggers

Q25. Define a trigger. Explain its types, uses and statement-level triggers.

Marks: 5 marks

Answer:

Definition: A trigger is a stored database program that executes automatically when a specified event occurs. Common DML events are INSERT, UPDATE and DELETE; supported systems may also offer triggers for DDL or database events.

Types:

- **By timing:** BEFORE, AFTER or INSTEAD OF (commonly used on views).
- **By level:** row-level triggers execute once per affected row; statement-level triggers execute once for the whole SQL statement.
- **By event:** DML triggers and, in supported systems, DDL or database-event triggers.

Statement-level trigger: it has no FOR EACH ROW clause and executes once even if the statement affects zero, one or many rows. It cannot use row-level :OLD and :NEW pseudorecords.

```
CREATE OR REPLACE TRIGGER trg_emp_stmt
BEFORE INSERT OR UPDATE OR DELETE ON employee
BEGIN
  dbms_output.put_line('DML executed on EMPLOYEE');
END;
/
```

Uses: triggers can enforce complex rules, record audit history, validate changes, generate automatic values and monitor database activity. Their code should remain focused because excessive trigger logic can be hard to debug.

PYQ coverage: A Q2e; C Q2d; D Q2A-iv

Q26. Create a PL/SQL trigger that audits INSERT, UPDATE and DELETE on Accounts. Explain its role.

Marks: 10 marks

Answer:

Audit table:

```
CREATE TABLE audit_log (
  account_id NUMBER,
  old_balance NUMBER,
  new_balance NUMBER,
  action_type VARCHAR2(10),
  action_time DATE
);
```

Trigger:

```
CREATE OR REPLACE TRIGGER trg_accounts_audit
AFTER INSERT OR UPDATE OR DELETE ON accounts
FOR EACH ROW
BEGIN
  IF INSERTING THEN
    INSERT INTO audit_log
    VALUES (:NEW.account_id, NULL, :NEW.balance,
            'INSERT', SYSDATE);
  ELSIF UPDATING THEN
    INSERT INTO audit_log
    VALUES (:OLD.account_id, :OLD.balance, :NEW.balance,
            'UPDATE', SYSDATE);
  ELSIF DELETING THEN
    INSERT INTO audit_log
    VALUES (:OLD.account_id, :OLD.balance, NULL,
            'DELETE', SYSDATE);
  END IF;
END;
/
```

Explanation: AFTER means it runs after a row-level DML operation. FOR EACH ROW makes it execute for every affected account. On INSERT, the old balance is NULL and new balance is recorded. On UPDATE, both old and new balances are recorded. On DELETE, old balance is recorded and new balance is NULL. SYSDATE records when the action occurred.

Benefits:

- The audit table preserves a traceable history of account changes.
- The action type and timestamp help investigate unauthorized or incorrect changes.
- Because the audit insert is in the same transaction, it normally commits or rolls back with the account change.
- Logging is centralized rather than separately implemented in each application.

Limitations: each change adds a write, the audit table needs retention/archiving, and privileges should prevent unauthorized users from altering or disabling the trigger.

PYQ coverage: A Q2e; B Q4c

---

9. DDL and DML Commands

Q27. Explain DDL and DML, state their roles and distinguish them with examples.

Marks: 5 marks

Answer:

Data Definition Language (DDL): defines or changes the structure of database objects such as tables, views and indexes. Main commands include CREATE, ALTER, DROP, TRUNCATE and RENAME. Example: CREATE TABLE Student(Roll INT PRIMARY KEY, Name VARCHAR(40));

Data Manipulation Language (DML): retrieves and changes the data stored inside tables. Common commands are SELECT, INSERT, UPDATE and DELETE. Example: UPDATE Student SET Name = 'Amit' WHERE Roll = 1;

Basis	DDL	DML
Purpose	Defines/changes schema and objects	Retrieves or modifies rows
Examples	CREATE, ALTER, DROP, TRUNCATE	SELECT, INSERT, UPDATE, DELETE

Basis	DDL	DML
Effect	Changes object definition	Changes table contents
Transactions	Commit/rollback behaviour depends on DBMS; often auto-committed	DML can generally be rolled back before COMMIT

Conclusion: DDL creates the framework in which data is stored, while DML operates on the data held within that framework.

PYQ coverage: A Q1d; B Q1c; D Q1iii

Q28. Explain and construct SQL for DELETE, TRUNCATE and DROP.

Marks: 5 marks

Answer:

DELETE: a DML command that removes selected rows or all rows while retaining the table structure. It supports a WHERE clause.

TRUNCATE: removes all rows but keeps the table structure. It normally does not support WHERE and is treated as DDL in the supplied answer key.

DROP: removes the table object, including its rows and its structure.

```
-- Delete selected rows only
DELETE FROM Student WHERE Roll = 3;

-- Remove all rows but keep the table
TRUNCATE TABLE Student;

-- Remove the table and its structure
DROP TABLE Student;
```

The key distinction is the effect: DELETE can filter rows; TRUNCATE clears the table; DROP removes the object itself. Rollback behaviour differs by DBMS, so use the course convention when writing an exam answer.

PYQ coverage: C Q1d

PART II — HIGH-PRIORITY TOPICS

10. Transactions and ACID Properties

Q29. What is a transaction? Explain the ACID properties with an example.

Marks: 5 marks

Answer:

Transaction: a set of logically related database operations treated as one unit of work. It should either complete fully or leave no lasting partial effect.

Consider transferring Rs 100 from Account A (Rs 600) to Account B (Rs 300):

```
R(A); A = A - 100; W(A);
R(B); B = B + 100; W(B);
COMMIT;
```

Atomicity: all operations take effect or none do. If failure happens after debiting A but before crediting B, the transfer is rolled back.

Consistency: the transaction takes the database from one valid state to another. Here, the total remains Rs 900 before and after the transfer.

Isolation: concurrent transactions should not see an incomplete intermediate state, such as A already debited while B is not yet credited.

Durability: after COMMIT, changes remain stored even if the server crashes. Together, ACID properties support reliable execution and protect database integrity.

PYQ coverage: B Q2f; C Q2e; D Q1v

Q30. Elaborate the transaction process with an example.

Marks: 5 marks

Answer:

A transaction starts when the DBMS begins processing a logical unit of work. Data operations can include READ, computation and WRITE. It then either commits successfully or is rolled back if an error occurs.

Example: transferring Rs 800 from account X to account Y.

```
R(X);
X = X - 800;
W(X);
R(Y);
Y = Y + 800;
W(Y);
COMMIT;
```

Steps:

- READ obtains the current value from the database into memory.
- The transaction calculates a new value, for example subtracting Rs 800 from X.
- WRITE records the changes for X and then Y.
- COMMIT makes the completed transfer permanent.
- If failure occurs before a successful commit, ROLLBACK restores old values and prevents a partial transfer.

If X is debited but the system fails before Y is credited, the account balances would be inconsistent. Transaction control and recovery ensure that either both operations succeed or the incomplete effects are undone.

PYQ coverage: C Q1f

11. Generalization and Specialization

Q31. Differentiate between generalization and specialization with suitable examples.

Marks: 5 marks

Answer:

Generalization is a bottom-up process. It combines lower-level entity types with common features into a superclass. Example: TIGER, LION and ELEPHANT can be generalized into ANIMAL, which stores shared attributes such as Name and Age.

Specialization is a top-down process. It divides a superclass into subclasses based on distinguishing characteristics. Example: EMPLOYEE(Emp_ID, Name, Salary) can be specialized into DEVELOPER(Programming_Language) and TESTER(Testing_Tool). The subclasses inherit the superclass identifier and common attributes.

Basis	Generalization	Specialization
Direction	Bottom-up	Top-down
Operation	Combines entity types	Divides an entity type
Focus	Common attributes	Distinctive attributes
Example	Tiger/Lion -> Animal	Employee -> Developer/Tester

Both represent an “is-a” relationship and are shown in EER models using a superclass/subclass structure. Depending on the rules, specialization may be disjoint or overlapping, and total or partial.

PYQ coverage: B Q3a; C Q2a; D Q1iv

12. SQL Constraints and Integrity Constraints

Q32. Describe the types of integrity constraints in SQL with examples.

Marks: 5 marks

Answer:

Integrity constraints are rules that restrict what data may be stored so that the database remains valid and consistent.

- **NOT NULL:** column must contain a value, e.g. Name VARCHAR(50) NOT NULL.
- **UNIQUE:** values must not be duplicated, e.g. Email VARCHAR(100) UNIQUE.
- **PRIMARY KEY:** uniquely identifies every row and is not NULL, e.g. Roll_No INT PRIMARY KEY.
- **FOREIGN KEY:** maintains referential integrity by requiring the value to match a referenced key or be NULL when allowed.
- **CHECK:** enforces a condition, e.g. Marks BETWEEN 0 AND 100.
- **DEFAULT:** supplies a value when none is provided, e.g. City DEFAULT 'Mumbai'.

Example:

```
CREATE TABLE Student (
 Roll_No INT PRIMARY KEY,
 Name VARCHAR(50) NOT NULL,
 Email VARCHAR(100) UNIQUE,
 Marks INT CHECK (Marks BETWEEN 0 AND 100),
 City VARCHAR(30) DEFAULT 'Mumbai',
 Dept_ID INT,
 FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);
```

These constraints reject invalid data at database level and help prevent duplicate identifiers, missing required values, invalid marks and references to nonexistent departments.

PYQ coverage: B Q2c; C Q2c; D Q2A-ii

PART III — MEDIUM-PRIORITY TOPICS

13. Lock-Based Protocols, Two-Phase Locking and Deadlock

Q33. Explain the Two-Phase Locking (2PL) protocol with an example. State advantages and disadvantages.

Marks: 10 marks

Answer:

Definition: lock-based concurrency control requires a transaction to obtain a lock before accessing a data item. A shared (S) lock is used for reading and may be held by multiple transactions. An exclusive (X) lock is required for writing and excludes incompatible locks.

Two phases:

- **Growing phase:** locks may be acquired, but none may be released.
- **Shrinking phase:** after releasing its first lock, the transaction may release locks but cannot acquire new ones.

The instant at which a transaction obtains its last lock is called its lock point.

Example: transfer Rs 50 from A to B.

```
lock-X(A); read A; A = A - 50; write A;
lock-X(B); read B; B = B + 50; write B;
unlock(A); unlock(B);
```

Both locks are acquired in the growing phase. After the first unlock, the transaction is in its shrinking phase and cannot acquire another lock.

Variants: Strict 2PL retains exclusive locks until commit/abort; Rigorous 2PL retains all locks until commit; Conservative 2PL obtains all required locks before starting and can prevent deadlock.

Advantages: guarantees conflict serializability, supports isolation and provides a clear locking discipline. **Disadvantages:** ordinary 2PL does not prevent deadlocks; locks reduce concurrency; transactions may wait or starve; basic 2PL may permit cascading rollbacks unless a stricter variant is used.

PYQ coverage: D Q3iii

Exam tip: For 10 marks include both phases, transaction example, variants, advantages and disadvantages.

Q34. Explain how lock-based protocols manage deadlocks and describe deadlock detection.

Marks: 5 marks

Answer:

Deadlock: two or more transactions wait for locks held by one another, so none can proceed. Example: T1 holds X(A) and waits for B, while T2 holds X(B) and waits for A.

Prevention: Conservative 2PL requests all required locks before starting; ordering data items may avoid circular wait. Timestamp schemes such as wait-die and wound-wait decide whether a transaction waits or is rolled back.

Detection: the DBMS constructs a wait-for graph. Each node is a transaction and an edge Ti -> Tj means Ti waits for a lock held by Tj. A cycle indicates deadlock.

Recovery: the system selects a victim transaction, rolls it back fully or partially, releases its locks and restarts it. Victim selection should avoid repeatedly aborting the same transaction.

Timeout: a transaction waiting beyond a chosen duration can be aborted. This is simple but can abort a transaction that was merely slow. Ordinary 2PL improves concurrency control but does not guarantee freedom from deadlock.

PYQ coverage: A Q1f

14. SQL Joins

Q35. Write SQL queries for INNER, LEFT, RIGHT and FULL OUTER JOIN and explain their results.

Marks: 5 marks

Answer:

Assume STUDENT(S_ID, Name, C_ID) and COURSE(C_ID, C_Name), linked by C_ID.

```
-- INNER JOIN: matching rows only
SELECT S.Name, C.C_Name
FROM STUDENT S INNER JOIN COURSE C
ON S.C_ID = C.C_ID;

-- LEFT JOIN: every student
SELECT S.Name, C.C_Name
FROM STUDENT S LEFT JOIN COURSE C
ON S.C_ID = C.C_ID;

-- RIGHT JOIN: every course
SELECT S.Name, C.C_Name
FROM STUDENT S RIGHT JOIN COURSE C
ON S.C_ID = C.C_ID;

-- FULL OUTER JOIN: unmatched rows on both sides
SELECT S.Name, C.C_Name
FROM STUDENT S FULL OUTER JOIN COURSE C
ON S.C_ID = C.C_ID;
```

Results: INNER JOIN displays only matching student-course pairs. LEFT JOIN also shows students without a course, with course fields as NULL. RIGHT JOIN includes every course even if there is no student. FULL OUTER JOIN retains unmatched rows from both tables and matches rows where possible.

MySQL note: MySQL does not directly support FULL OUTER JOIN; the result can be emulated using a combination of LEFT JOIN and RIGHT JOIN with UNION, avoiding duplicate matched rows.

PYQ coverage: A Q2d

Q36. Differentiate between natural join and equi-join using SQL queries.

Marks: 5 marks

Answer:

Equi-join: explicitly specifies an equality condition between two relations. If SELECT * is used, both join columns may appear in the output.

Natural join: automatically matches columns with the same name and returns one copy of each common column. It may be risky if unrelated columns share a name.

```
-- Equi-join
SELECT *
FROM STUDENT S JOIN COURSE C
ON S.C_ID = C.C_ID;

-- Natural join
SELECT *
FROM STUDENT NATURAL JOIN COURSE;
```

Basis	Equi-join	Natural join
Condition	Explicit equality in ON/WHERE	Automatically uses same-named attributes
Common columns	Both join columns may appear	Common join column appears once
Control	More explicit	Depends on column names

Both combine related records. For full marks, include each query and explain how C_ID links student and course rows.

PYQ coverage: B Q3c

---

15. Stored Procedures

Q37. Define a stored procedure in PL/SQL. Explain its advantages and write an example.

Marks: 5–10 marks

Answer:

Definition: a stored procedure is a named PL/SQL subprogram stored and compiled in the database. It can accept parameters and execute SQL and procedural statements on demand. A procedure performs an action; unlike a function, it does not have to return a value directly.

Structure: declaration section is optional, BEGIN...END contains executable statements, and EXCEPTION can handle runtime errors. Parameter modes include IN, OUT and IN OUT.

Example — salary raise:

```
CREATE OR REPLACE PROCEDURE raise_salary
  (p_emp_id IN NUMBER, p_percent IN NUMBER) AS
BEGIN
  UPDATE employee
  SET salary = salary + salary * p_percent / 100
  WHERE emp_id = p_emp_id;
END raise_salary;
/

EXECUTE raise_salary(101, 10);
```

Advantages:

- **Reuse and modularity:** logic is written once and called from several applications.
- **Performance:** compiled server-side logic can reduce repeated parsing and network round trips.
- **Security:** users may receive execute permission without direct table access.
- **Consistency:** the same business rules apply to every caller.
- **Maintainability:** common logic can be changed in one database object.

PYQ coverage: A Q1e; B Q4c

---

16. DBMS Architecture

Q38. Define DBMS architecture and explain its key components.

Marks: 5 marks

Answer:

DBMS architecture describes the components that accept user requests, process queries, manage data and store database information.

- **Users and interfaces:** application users, programmers, sophisticated users, specialized users and the DBA.
- **Query processor:** DDL interpreter records schema definitions; DML compiler translates queries; query optimizer selects an efficient plan; query evaluation engine executes it.
- **Storage manager:** manages database files, buffers, authorization, integrity and transactions. Components include file manager, buffer manager and transaction manager.
- **Disk storage:** contains data files, indexes and the data dictionary/metadata describing database objects.

Working: the user submits a query through an interface. The query processor translates and optimizes it. The storage manager accesses or changes records while enforcing permissions and transaction rules, and then returns the result. Separating these responsibilities makes database processing easier to manage, optimize and secure.

PYQ coverage: A Q1a

17. Three-Tier DBMS Architecture

Q39. Explain the three-tier architecture of DBMS with advantages and disadvantages.

Marks: 5 marks

Answer:

Presentation/client tier: provides the user interface, such as a browser or mobile/desktop application. It sends requests and displays results.

Application/business tier: processes client requests, applies business rules and validation, manages application logic and communicates with the database. It may handle authentication and load distribution.

Database tier: contains the DBMS and stored data. It executes queries, applies constraints and manages transactions.

Request flow: Client -> Application Server -> Database Server. The response travels back through the application layer to the client.

Advantages: better scalability, central business logic and improved security because clients do not access the database directly. **Disadvantages:** greater development/maintenance complexity and extra communication overhead. Three-tier architecture is common in multi-user web applications.

PYQ coverage: C Q1a

18. SQL vs PL/SQL

Q40. Differentiate between SQL and PL/SQL.

Marks: 4–5 marks

Answer:

Basis	SQL	PL/SQL
Nature	Declarative: describes what operation is required	Procedural extension; defines a sequence of steps
Execution	Usually one SQL statement at a time	Can execute a block of statements
Programming constructs	No general procedural IF/loop variables in a plain SQL statement	Supports variables, IF, loops, cursors and exceptions
Program units	Queries and data commands	Procedures, functions, triggers and packages
Use	Define, retrieve and manipulate data	Implement database-side application logic

SQL is used to query and manipulate relational data. PL/SQL is Oracle's procedural extension that embeds SQL inside larger programs. PL/SQL supports variables, conditions, loops and exception handling, making it useful when multiple database operations must execute together.

PYQ coverage: B Q1e; C Q1e

19. Types of Attributes in the ER Model

Q41. Explain the types of attributes in an ER diagram with examples.

Marks: 5 marks

Answer:

An attribute is a property describing an entity. For a STUDENT entity, attributes may be of several types:

- **Simple/atomic:** cannot be divided further, e.g. Roll_No or Gender.
- **Composite:** can be split into components, e.g. Address into Street, City, State and Country.
- **Single-valued:** one value for each entity, e.g. Date_of_Birth.
- **Multivalued:** several values may be stored for one entity, e.g. Phone_No. Traditional ER notation uses a double ellipse.
- **Derived:** calculated from another attribute, e.g. Age from Date_of_Birth. It is often represented by a dashed ellipse.
- **Key:** uniquely identifies an entity, e.g. Roll_No, often underlined.

Applied example: BOOK(ISBN, Title, Authors, Publisher, Year_of_Publication, Age_of_Book). ISBN is the key, Title is simple, Authors may be multivalued, Publisher may be composite and Age_of_Book is derived. Identifying attribute types helps build the ER model and determines how attributes map to tables.

PYQ coverage: B Q1a; C Q2b

20. SQL Aggregate Functions

Q42. Write SQL queries for COUNT, SUM, AVG, MAX and MIN in an employee database. Explain them.

Marks: 5 marks

Answer:

Aggregate functions calculate one value from a set of rows. Assume EMPLOYEE(Emp_ID, Name, Dept, Salary).

```
-- Number of employees
SELECT COUNT(*) FROM employee;

-- Number of distinct departments
SELECT COUNT(DISTINCT dept) FROM employee;

-- Total salaries
SELECT SUM(salary) FROM employee;

-- Average salary
SELECT AVG(salary) FROM employee;

-- Highest and lowest salary
SELECT MAX(salary) FROM employee;
SELECT MIN(salary) FROM employee;
```

Function	Meaning
COUNT(*)	Counts all rows.
COUNT(column)	Counts non-NULL values in the column.
SUM(column)	Adds non-NULL numeric values.
AVG(column)	Average of non-NULL numeric values.
MAX / MIN	Largest / smallest value.

Aggregates can be grouped. Example: SELECT Dept, AVG(Salary) FROM Employee GROUP BY Dept; calculates one average per department. WHERE filters rows before grouping; HAVING filters groups after aggregation.

PYQ coverage: B Q3f; C Q3c-v

PART IV — LOW-PRIORITY BUT EXAMINABLE TOPICS

21. Recovery System

Q43. Explain the steps taken by a recovery system after a crash following a partially committed transaction.

Marks: 5 marks

Answer:

Purpose: recovery restores the database to a consistent state after failure and helps preserve atomicity and durability.

Log-based recovery: updates are recorded in a log containing details such as transaction, data item, old value and new value. Write-ahead logging requires the relevant log record to reach stable storage before the corresponding data page is written.

Steps after restart:

- Start scanning from the most recent checkpoint to reduce recovery work.
- Classify transactions: a durable commit record means the transaction committed; a transaction without a commit record is incomplete.
- UNDO incomplete transactions using old values, normally scanning the log backwards.
- REDO committed transactions using new values where needed, normally scanning forwards.
- Resume normal processing after the database has been returned to a consistent state.

A transaction that reached partially committed state but has no durable commit record is treated as incomplete and is undone. This prevents partial transaction effects from remaining in the database.

PYQ coverage: A Q2f

22. Transaction States

Q44. Illustrate transaction states with a proper diagram and explain each state.

Marks: 5 marks

Answer:

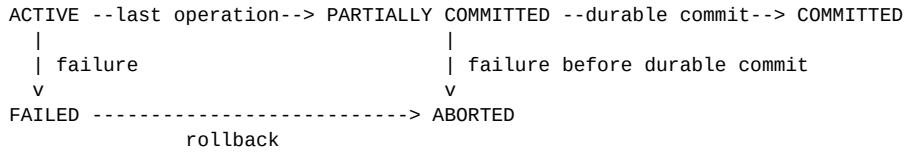
Active: the transaction is executing read/write operations; its outcome is not yet permanent.

Partially committed: the last operation has executed, but the system has not yet made all changes durable.

Committed: the transaction succeeded and its changes are permanent.

Failed: a failure means normal execution cannot continue, for example due to an error or system problem.

Aborted: the transaction has been rolled back and the database returned to the state before it. The system may restart or terminate it.



The successful path is Active -> Partially Committed -> Committed. If a failure occurs, Active or Partially Committed can go to Failed, followed by Aborted after rollback.

PYQ coverage: B Q3d

23. Cursors and Functions in PL/SQL

Q45. Explain functions and cursors in PL/SQL with examples.

Marks: 5 marks

Answer:

Function: a named PL/SQL subprogram that must return a value using RETURN. It may accept parameters and can be called from PL/SQL expressions.

```
CREATE OR REPLACE FUNCTION totalCustomers
RETURN NUMBER IS
  total NUMBER := 0;
BEGIN
  SELECT COUNT(*) INTO total FROM customers;
  RETURN total;
END;
/
```

Cursor: a cursor points to the context area holding a SQL statement's result set, allowing PL/SQL to process rows. An implicit cursor is created automatically for DML and single-row SELECT INTO operations. An explicit cursor is declared by the programmer for a query returning several rows.

Explicit cursor steps: DECLARE the cursor query, OPEN it, FETCH each row into variables, and CLOSE it when complete.

```
DECLARE
  CURSOR c_emp IS SELECT emp_name, salary
    FROM employee WHERE salary > 30000;
  v_name employee.emp_name%TYPE;
  v_sal employee.salary%TYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_name, v_sal;
    EXIT WHEN c_emp%NOTFOUND;
    dbms_output.put_line(v_name || ' earns ' || v_sal);
  END LOOP;
  CLOSE c_emp;
END;
/
```

Cursor attributes such as %FOUND, %NOTFOUND, %ROWCOUNT and %ISOPEN report cursor status. A function returns one value, whereas a cursor supports row-by-row processing of a query result.

PYQ coverage: B Q2e

---

24. Total and Partial Participation

Q46. Explain total and partial participation in an ER model using a college database example.

Marks: 5 marks

Answer:

Participation constraints specify whether every entity in an entity set must participate in a relationship.

Total participation: every entity must participate in at least one relationship instance. It is shown by a double line. Example: if every student must enrol in at least one course, STUDENT has total participation in ENROLLS.

Partial participation: only some entities participate. It is shown by a single line. Example: a course may currently have no enrolled students, so COURSE has partial participation in ENROLLS.

Second example: if every department must have a head, DEPARTMENT has total participation in HEADS; if only some professors are heads, PROFESSOR has partial participation.

Importance: participation expresses mandatory business rules and affects schema mapping. Total participation may permit a NOT NULL foreign key on the appropriate side. If both sides of a 1:1 relationship are partial, merging tables may create many NULLs, so separate tables may be preferable.

PYQ coverage: B Q2a

---

25. Database Administrator Responsibilities

Q47. List five DBA responsibilities and explain problems caused when they are not performed properly.

Marks: 5 marks

Answer:

- **Database design and implementation:** defines schemas, tables and relationships. Poor design causes redundancy, anomalies and inconsistent data.
- **Security management:** manages users, privileges and access controls. Neglect may permit unauthorized reading, alteration or theft of data.
- **Backup and recovery:** plans and tests backups and restores. Failure can cause permanent data loss and long downtime after a crash.
- **Performance monitoring and tuning:** monitors queries, indexes and configuration. Poor tuning causes slow queries and bottlenecks.
- **Troubleshooting, capacity planning and maintenance:** handles failures, storage growth, migrations, patches and documentation. Neglect can cause outages, storage shortages or failed upgrades.

Other duties include enforcing integrity constraints, supporting users, coordinating with developers and planning upgrades. A DBA helps keep the database secure, available, consistent and performant.

PYQ coverage: D Q1i

---

26. Super Key and Candidate Key

Q48. Explain super key and candidate key with a suitable example.

Marks: 4–5 marks

Answer:

Super key: a set of one or more attributes that uniquely identifies every tuple in a relation. It may contain extra attributes that are not needed for uniqueness.

Candidate key: a minimal super key; no proper subset can uniquely identify every tuple.

Example: Consider STUDENT(Roll, Name, Gender, Age, Address). Assume Roll is unique. Then {Roll} is a candidate key. {Roll, Name} is a super key but not a candidate key because Roll alone is sufficient. If Email is also guaranteed unique and non-NULL, {Email} can be another candidate key.

Related terms: the designer chooses one candidate key as the primary key. Other candidate keys are alternate keys. A foreign key references a key in another relation. A composite key contains two or more attributes.

Relationship: every candidate key is a super key, but not every super key is a candidate key. Candidate keys are minimal; super keys need not be.

PYQ coverage: B Q1b

FINAL EXAM-WRITING CHECKLIST

- For normalization, state the key and functional dependencies, identify the exact violation, and show the decomposed relations.
- For FD problems, compute attribute closure before choosing a candidate key or judging normal form.
- For relational algebra, explain each operator and show a syntactically clear expression.
- For ER questions, include entity names, attributes, keys, relationships and cardinalities. Convert M:N relationships into separate tables.
- For SQL, write complete statements with aliases and join conditions. State assumptions if the question schema is missing a needed column.
- For serializability, list conflicts, build the precedence edges, and state whether a cycle exists.
- For triggers, distinguish row-level from statement-level and use :OLD, :NEW, INSERTING, UPDATING and DELETING correctly.
- For 10 marks, add working, code, schema or diagram and a conclusion; do not stop after a definition.
- For diagrams, underline keys and label 1:1, 1:N or M:N cardinalities clearly.

Source basis: Supplied DBMS Previous-Year Question Bank and DBMS Answer Key. The ranking and topic wording follow those files; model answers have been expanded into answer-sheet form.